

ENPM665 Endterm Group Project

Cloud Security Hardening and Improvements

Group 15

<u>Name</u>	<u>UID</u>
Raghava Gatadi	raghavag/121941190
Piyush Anand	panand14/121327543
Miles Levine	mlevin56/117903881
Balakrishna Nair Nalivalapil	bkrishna/121179722
Chethana Datta Jugulam	cdatta/122261464
Mohammed Emad Sultan	mesultan/121227461

1. Executive Summary

1.1 Project Overview

Following the comprehensive security assessment conducted during the midterm phase, Group 15 identified critical vulnerabilities within the OWASP Juice Shop cloud infrastructure. The assessment revealed a fragile, permissive architecture characterized by unencrypted storage, hardcoded credentials, and unrestricted network access. This project serves as the remediation phase, transitioning the environment from a high-risk state to a hardened, defense-in-depth architecture.

Our primary objective was to redesign the data handling and network boundaries of the application to align with AWS Well-Architected best practices and NIST security frameworks. By leveraging AWS-native services such as AWS Key Management Service (KMS), Secrets Manager, and Web Application Firewall (WAF), we successfully mitigated high-severity risks related to data theft, credential compromise, and unauthorized network intrusion.

1.2 Selected Focus Areas

To address the most immediate threats to the confidentiality and integrity of the workload, the group prioritized the following two focus areas for the core report:

1. Data Protection and Encryption:

Selected due to the critical nature of the findings "DS-01: Unencrypted EC2 Root Volume" and "DS-02: Hardcoded Database Credentials." These vulnerabilities represented a direct violation of HIPAA and NIST safeguards regarding data at rest and credential hygiene.

2. Network and Access Control Security:

Selected to remediate "NW-01: Open SSH Access" and "NW-02: Direct Application Exposure." The midterm assessment found that the application was directly exposed to the public internet without Layer-7 filtering, making it highly susceptible to brute-force attacks and botnet scanning.

1.3 Summary of Improvements

The group implemented four key security improvements across the selected focus areas to enforce a stronger security posture:

Data Protection & Encryption Improvements:

- **Enabled EBS Volume Encryption:** We modified the CloudFormation template to enforce AES-256 encryption on the EC2 root volume using AWS KMS (**alias/aws/ebs**). This ensures that all data at rest, including system logs and application files, is rendered unreadable to unauthorized parties, mitigating the risk of disk theft or snapshot leakage.
- **Implemented AWS Secrets Manager:** We eliminated the use of plaintext credentials in the UserData bootstrap script. The MariaDB root password was migrated to a secure vault in AWS Secrets Manager (**enpm665/juice/dbrootpassword**), and the EC2 instance was granted a least-privilege IAM role to retrieve the secret dynamically at runtime.

Network and Access Control Improvements:

- **Restricted SSH Access:** We hardened the EC2 Security Group by removing the rule allowing global SSH access (**0.0.0.0/0**). Inbound traffic on port 22 is now strictly limited to a single trusted Administrator IP, reducing the remote management attack surface by nearly 100%.
- **Deployed CloudFront and AWS WAF:** We removed direct public access to the application port (3000) and routed all traffic through an Amazon CloudFront distribution. We further secured this distribution by attaching AWS WAF rules that enforce Geo-Blocking (allowing only U.S. traffic) and IP-based blocking for known malicious actors.

1.4 Additional Measures and Advanced Defenses

Beyond the foundational hardening of the compute and data layers, the team implemented three specific advanced controls to address business continuity gaps and sophisticated Layer 7 threats. These measures provide a robust defense-in-depth strategy that exceeds the baseline requirements:

- **Disaster Recovery (DR) Solution:** To remediate the high risk of permanent data loss, we implemented a centralized AWS Backup solution. This defined a 24-hour Recovery Point Objective (RPO) and was validated through a full-cycle "Game Day" simulation where a compromised instance was successfully restored from an encrypted recovery vault.
- **Geo-Blocking (Allow Only U.S. Traffic):** We deployed a geographic filtering rule within AWS WAF to restrict application access exclusively to users within the United States. This significantly reduced the attack surface by automatically blocking traffic from high-risk foreign regions often associated with botnets and automated scanning networks.
- **IP-Based Deny Rule for Known Malicious Addresses:** We established a proactive blocking mechanism using AWS WAF to deny requests from specific IP addresses identified as malicious. This allows the security team to manually blacklist attackers or bot networks that attempt to probe the application, ensuring that hostile traffic is stopped at the network edge before reaching the infrastructure.

2. Background: Connection to Midterm

During the midterm security assessment of the OWASP Juice Shop e-commerce platform, Group 15 conducted a comprehensive review of the AWS infrastructure. This assessment revealed a fragile architecture that prioritized ease of deployment over security, resulting in significant gaps in confidentiality, integrity, and availability. The findings from that report directly informed the remediation strategy executed in this final project.

2.1 Overview of Midterm Weaknesses

The midterm analysis highlighted that the Juice Shop application relied on a monolithic, single-instance architecture hosted in a public subnet. The environment lacked the "defense-in-depth" controls necessary to protect modern cloud workloads.

Key architectural failures included:

- **Single Point of Failure:** The architecture relied heavily on a single EC2 instance ("VulnerableInstance") with no automated backup or recovery mechanisms, creating a high risk of permanent data loss.
- **Permissive Network Access:** Security groups were configured with overly broad permissions, allowing global access to administrative ports and exposing the application server directly to the internet without edge protection.
- **Lack of Encryption:** Sensitive data, including the database file system and administrative credentials, existed in plaintext, violating standard compliance frameworks such as HIPAA and NIST.

These weaknesses collectively created an environment where a single misconfiguration or compromised credential could lead to a total system breach.

2.2 Findings Related to Data Protection

The Data Protection focus area was selected to address two critical high-severity vulnerabilities identified in the Data Security (DS) section of the midterm report. These findings represented immediate threats to data confidentiality.

Finding DS-01: Unencrypted EC2 Root Volume

The midterm assessment revealed that the EC2 instance storing application and database files operated with an unencrypted EBS root volume.

- **The Risk:** This configuration left all data at rest vulnerable. If the physical disk were compromised or if an unencrypted snapshot were inadvertently shared or leaked, an attacker could mount the volume and read the file system without restriction.
- **Compliance Gap:** This directly violated HIPAA encryption requirements and NIST 800-53 controls (SC-28) regarding the protection of data at rest.
- **Connection to Project:** This finding necessitated the implementation of Improvement #1, which enforces AES-256 encryption using AWS KMS for the instance root volume.

Finding DS-02: Hardcoded Plaintext MariaDB Password

The review of the CloudFormation templates and system logs showed that the database root password was hardcoded directly into the EC2 UserData provisioning script.

- **The Risk:** The password was visible in plaintext within the CloudFormation template, the AWS Console, and the system log file (`/var/log/cloud-init-output.log`) on the instance. Anyone with read access to the console or the instance could escalate privileges to full database administrator.
- **Connection to Project:** This critical flaw drove the implementation of Improvement #2, which removed the static password and replaced it with dynamic retrieval via AWS Secrets Manager.

2.3 Findings Related to Network Security

The Network Security focus area was chosen to remediate the excessive attack surface exposure identified in the Network (NW) section of the midterm report. The assessment found that the network perimeter was virtually nonexistent.

Finding NW-01: Open SSH Access (Port 22)

The Security Group attached to the application server contained an inbound rule allowing SSH traffic on **Port 22** from **0.0.0.0/0** (Anywhere).

- **The Risk:** This configuration exposed the administrative interface to the entire internet, subjecting the instance to continuous automated brute-force attacks, credential stuffing, and potential exploitation of SSH protocol vulnerabilities.
- **Connection to Project:** This finding led to Improvement #1 in the Network focus area, which involved restricting SSH access to a single trusted Administrator IP address to enforce least-privilege access.

Finding NW-02: Direct Application Exposure (No Layer 7 Filtering)

The midterm report noted that users and attackers accessed the application directly via the EC2 instance's public IP on Port 3000 (**`http://<EC2-IP>:3000`**).

- **The Risk:** There was no Web Application Firewall (WAF) or Content Delivery Network (CDN) in place. This meant that malicious traffic, including SQL Injection attempts, Cross-Site Scripting (XSS) probes, and bot scanners, reached the application server directly without inspection or filtering.
- **Connection to Project:** To resolve this, Improvement #2 was implemented to route all traffic through Amazon CloudFront and AWS WAF, effectively obscuring the origin server and filtering malicious requests at the edge.

3. Objectives

This project focuses on strengthening the security posture of the OWASP Juice Shop deployment by improving both data-level confidentiality and network-level access control. Each objective defined below directly addresses a specific finding from the Midterm assessment and contributes to reducing the exposure, exploitability, and attack surface of the application.

3.1 Focus Area 1: Data Protection & Encryption

Objective:

The primary objective of this focus area is to ensure the confidentiality of system and application data at rest and to enforce strict credential hygiene. By transitioning from plaintext storage to cryptographic protection, we aim to align the infrastructure with HIPAA and NIST compliance standards regarding data privacy.

Targeted Improvements:

1. Enable EBS Encryption for the EC2 Root Volume

- **The Goal:** To move from an unencrypted storage state to fully enforced AES-256 encryption using AWS Key Management Service (KMS).
- **Risk Addressed:** In the Midterm, the EC2 root volume was unencrypted (Finding DS-01). This meant that if a snapshot was leaked or the disk was physically compromised, all file system data—including logs and application source code—would be readable.

- **Implementation Strategy:** We successfully modified the CloudFormation template to enforce encryption at the time of resource provisioning (BlockDeviceMappings). This ensures that the operating system and data are never written to the disk in plaintext.

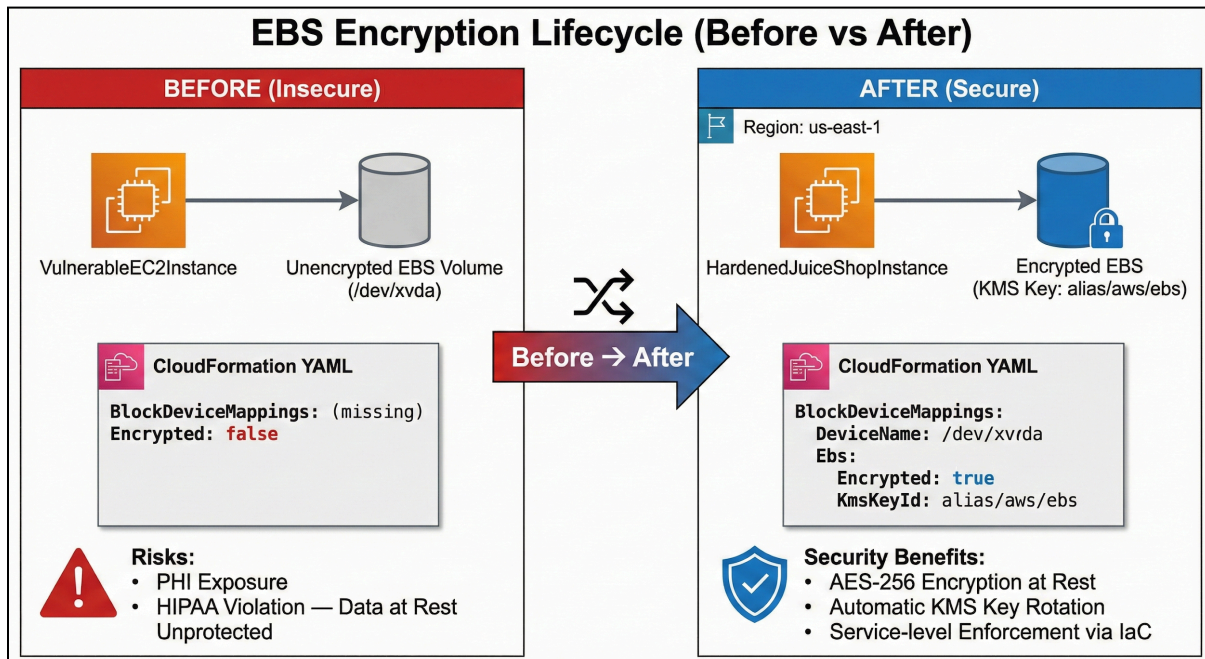


Figure 1: EBS Encryption Lifecycle—Comparison of Unencrypted vs. KMS-Encrypted EC2 Root Volumes

2. Secure MariaDB Root Password using AWS Secrets Manager

- **The Goal:** To eliminate the use of hardcoded credentials in infrastructure code and enable dynamic, auditable secret retrieval.
- **Risk Addressed:** The Midterm identified that the database root password was embedded in plaintext within the EC2 UserData script (Finding DS-02). This exposed administrative credentials to anyone with access to the AWS Console or system logs.
- **Implementation Strategy:** We replaced the static password with a secure secret stored in AWS Secrets Manager. The EC2 instance now uses a least-privilege IAM role to retrieve this secret programmatically during the boot process, ensuring credentials never appear in code repositories or logs.

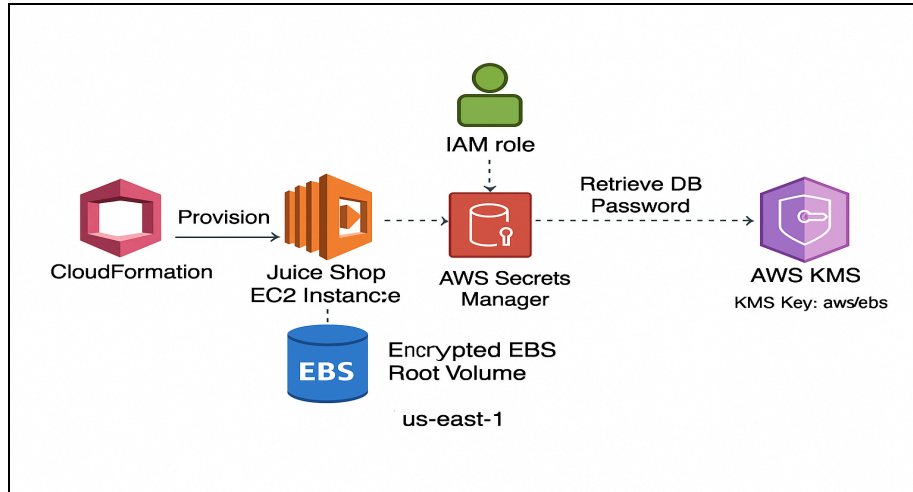


Figure 2: Secrets Manager Credential Retrieval Flow for Secure MariaDB Initialization

3.2 Focus Area 2: Network & Access Control Security

Objective:

The objective of this focus area is to minimize the external attack surface by enforcing "least-privilege" network access. We aim to shield the application server from direct internet exposure and ensure that administrative access is strictly limited to authorized personnel.

Targeted Improvements:

1. Restrict SSH Access to Single Trusted Administrator IP

- **The Goal:** To eliminate the risk of automated brute-force attacks by closing the global SSH port.
- **Risk Addressed:** The Midterm revealed that Port 22 was open to the entire internet (0.0.0.0/0) (Finding NW-01). This configuration invited constant scanning and credential stuffing attacks from malicious botnets.
- **Implementation Strategy:** We updated the EC2 Security Group to remove the global allow rule. Inbound SSH traffic is now restricted to a single, verified Administrator IP address (/32 CIDR). Any connection attempt from an unrecognized IP is immediately dropped by the AWS firewall.

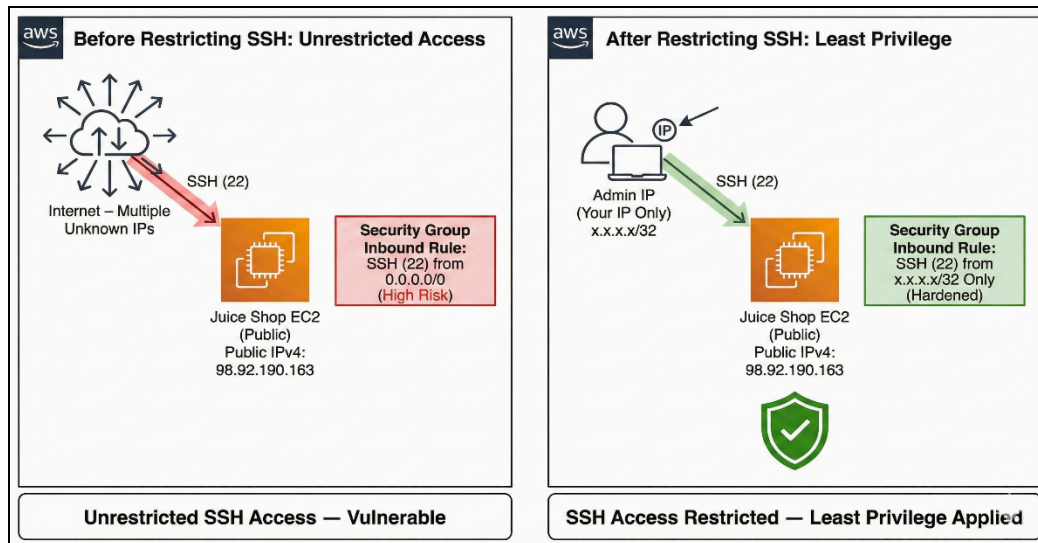


Figure 3: Before and After Restricting SSH Access

2. Route Application Traffic via CloudFront (Removing Direct EC2 Exposure)

- **The Goal:** To obscure the backend infrastructure and enforce Layer-7 traffic inspection before requests reach the server.
- **Risk Addressed:** The Midterm showed that the application was accessed directly via the EC2 public IP on Port 3000 (Finding NW-02). This direct exposure meant there was no edge protection against DDoS attacks or web exploits.
- **Implementation Strategy:** We deployed an Amazon CloudFront distribution to serve as the public-facing entry point. We configured the EC2 Security Group to deny direct public HTTP access, forcing all traffic to pass through CloudFront. This architecture not only hides the origin server's IP address but also allows us to apply AWS WAF rules at the edge.

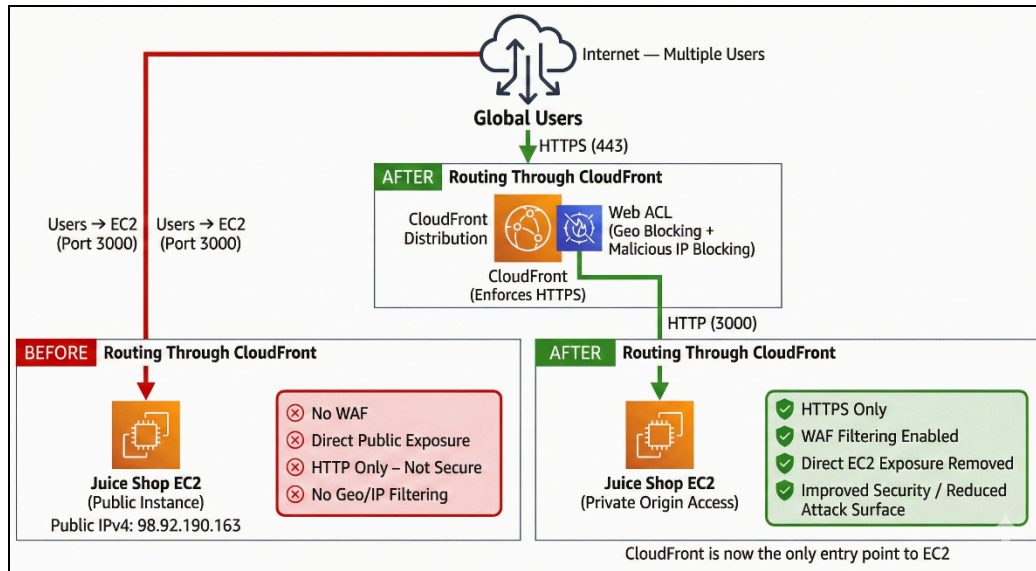


Figure 4: EC2 Security Group Hardening and CloudFront/WAF Implementation

4. Implementation Plan

This section details the exact technical steps used to implement the four selected improvements across both focus areas. Each subsection includes AWS configurations, specific parameters used, and references to the architecture diagrams and evidence screenshots.

4.1 Focus Area 1: Data Protection & Encryption

Improvement 1: Enable EBS Encryption for EC2 Root Volume

Objective:

Enforce encryption at the infrastructure provisioning layer to ensure all data at rest is protected by AWS KMS.

Step A: Identification of Insecure EBS Configuration (Before State)

We reviewed the existing EC2 root volume in the AWS Console. As identified in the Midterm, the volume state showed **Encryption: Not Encrypted** and **KMS Key ID: None**. This confirmed that the file system was exposed.

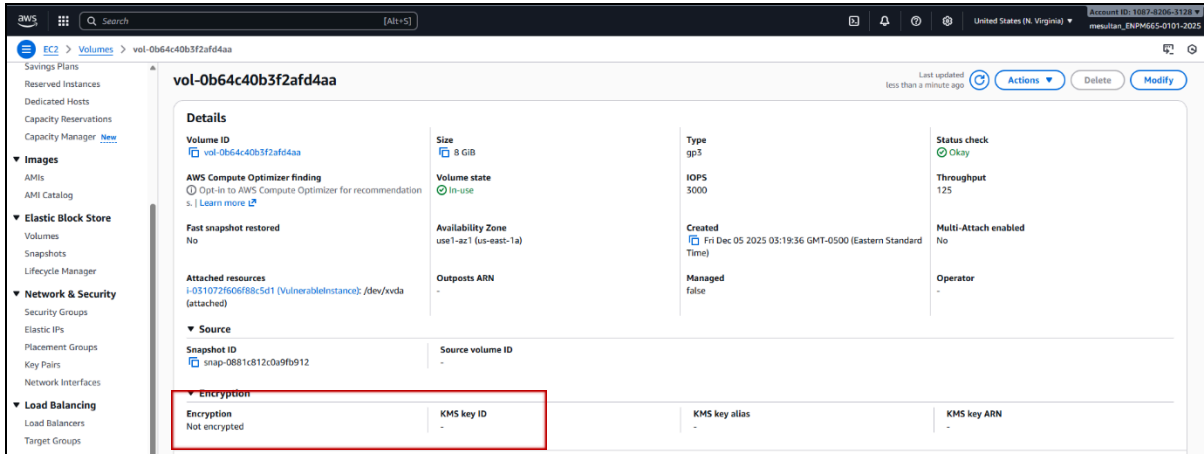


Figure 5: EBS Root Volume Before Encryption (Unencrypted)

i-031072f606f88c5d1 (VulnerableInstance)							
Block devices							
Volume ID	Device name	Volume size (GiB)	Volume State	Attachment status	Attachment time	Encrypted	KMS key ID
vol-0b64c40b3f2afd4aa	/dev/xvda	8	In-use	Attached	2025/12/05 03:19 GMT-5	No	-

Figure 6: Block Devices Storage Before Encryption (Unencrypted)

Step B: Updates to CloudFormation BlockDeviceMappings

To remediate this, we modified the CloudFormation template used to launch the **VulnerableEC2Instance**. We added a **BlockDeviceMappings** section to explicitly enforce encryption properties.

Step C: KMS Key ID Integration

We configured the template to use the AWS-managed KMS key alias (alias/aws/ebs) for seamless key management.

Configuration Snippet Applied:

```
BlockDeviceMappings:
  - DeviceName: "/dev/xvda"

  Ebs:
    VolumeSize: 20
    VolumeType: gp3
    Encrypted: true
    KmsKeyId: alias/aws/ebs
```

Step D: Redeployment Methodology

We executed an **Update Stack** action within CloudFormation. This triggered the replacement of the EC2 instance. The new instance launched with the encryption properties defined in the template, ensuring no unencrypted blocks were ever written to the disk.

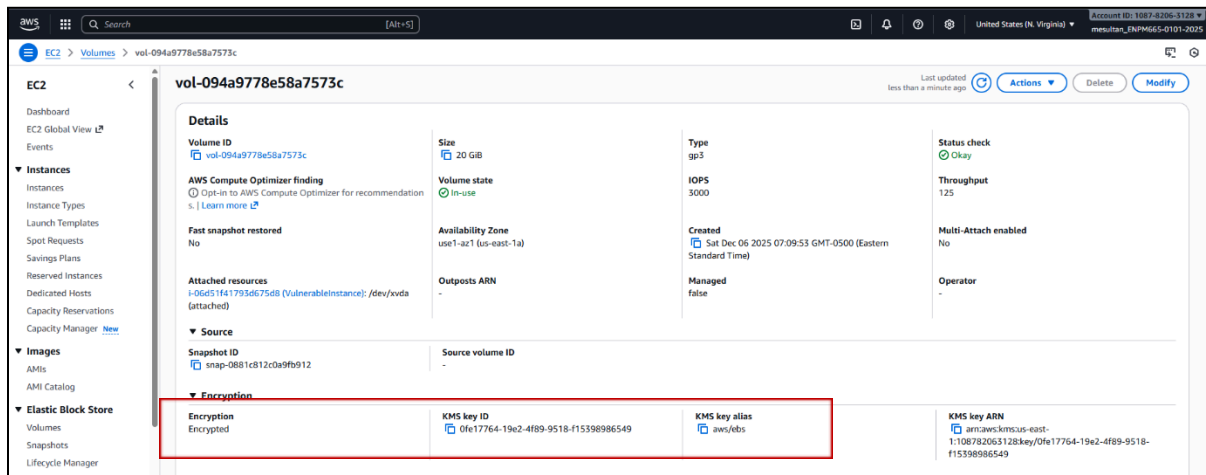


Figure 7: EBS Root Volume After Encryption Enabled

```
PS C:\Users\mdema\downloads> aws ec2 describe-volumes --region us-east-1 --volume-ids vol-094a9778e58a7573c --query "Volumes[*].Encrypted"
[
  true
]
PS C:\Users\mdema\downloads> |
```

Figure 8: CLI Validation of Encrypted Volume

Improvement 2: Secure MariaDB Password with Secrets Manager

Objective:

Remove hardcoded plaintext credentials from the UserData script and implement dynamic secret retrieval.

Step A: Creation of Secure Secret

We navigated to AWS Secrets Manager and created a new secret to store the database root password securely.

- **Secret Name:** enpm665/juice/dbrootpassword
- **Encryption:** Protected by AWS KMS.
- **Value:** The strong password string required for MariaDB initialization.

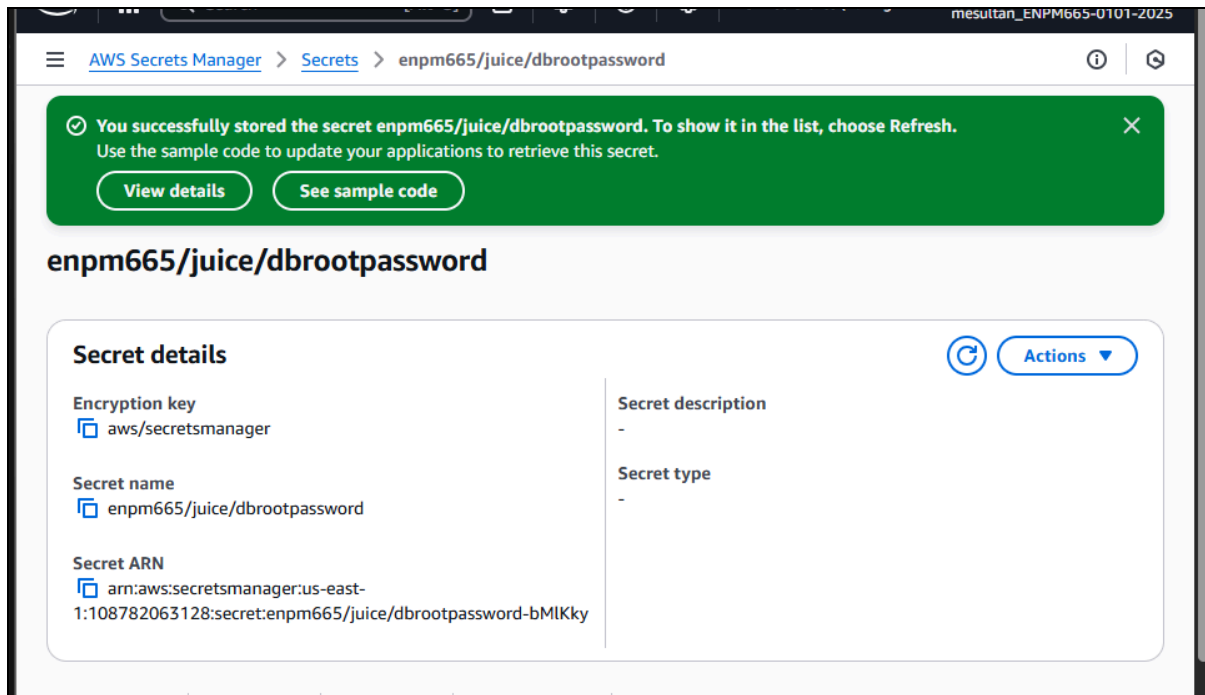


Figure 9: Secret Stored in AWS Secrets Manager

Step B: IAM Role Configuration (Least Privilege)

To allow the EC2 instance to retrieve this secret without using static AWS Access Keys, we created a specific IAM Role.

- **Role Name:** enpm665-juice-ec2
- **Policy:** We attached an inline policy allowing only `secretsmanager:GetSecretValue` specifically for the ARN of the secret created in Step A.

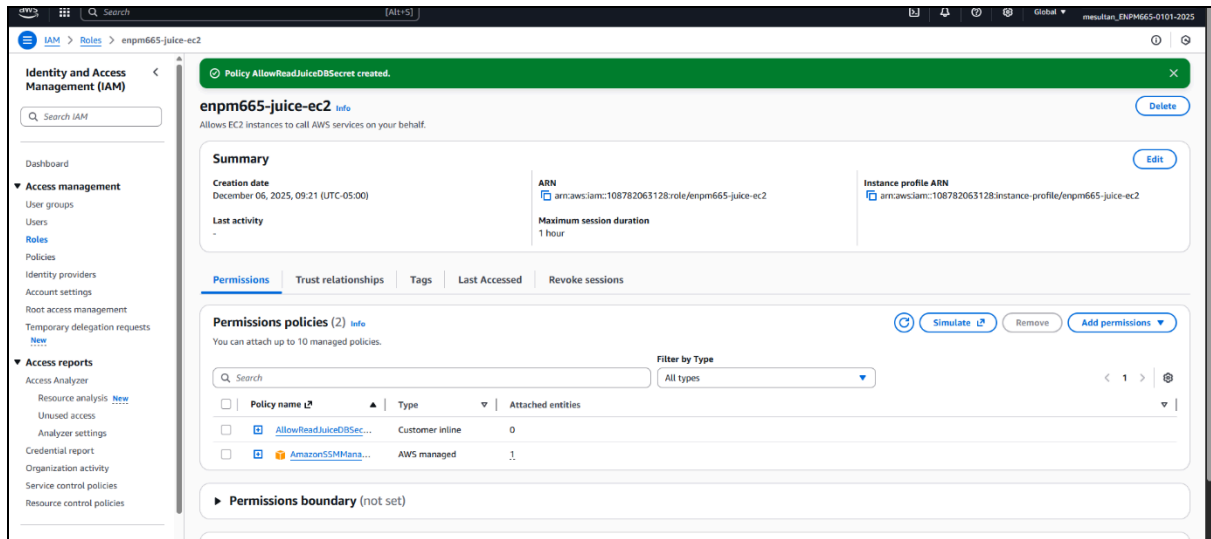


Figure 10: EC2 IAM Role With SecretsManager Permissions

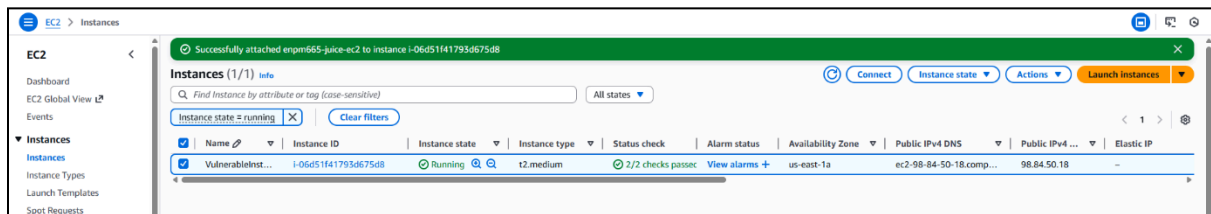


Figure 11: EC2 IAM Role attached to the EC2 instance successfully

Step C: Refactoring UserData Script

We completely rewrote the UserData section of the CloudFormation template.

1. **Removed:** The hardcoded line password = 'vulnerable'.
2. **Added:** Logic to install jq (JSON processor) and awscli.
3. **Added:** A command to fetch the secret value dynamically at runtime.

Updated Script Logic:

```
# Retrieve DB password from Secrets Manager
SECRET_JSON=$(aws secretsmanager get-secret-value \
--secret-id enpm665/juice/dbrootpassword \
--query SecretString \
--output text \
--region us-east-1)

DB_ROOT_PASSWORD=$(echo "$SECRET_JSON" | jq -r .dbrootpassword)
```

```

value: !Ref InstanceName
UserData:
  Fn::Base64:
    !Sub |
      #!/bin/bash
      LOG_FILE="/var/log/user_data.log"

      # Update the instance
      echo "Updating the instance" | tee -a $LOG_FILE
      yum update -y 2>&1 | tee -a $LOG_FILE

      # Install required software (Node.js, NPM, and MariaDB)
      echo "Installing Node.js and MariaDB" | tee -a $LOG_FILE
      curl -sL https://rpm.nodesource.com/setup_24.x | bash - 2>&1 | tee -a $LOG_FILE
      yum install -y nodejs mariadb105-server.x86_64 git 2>&1 | tee -a $LOG_FILE

      # Start and enable MariaDB
      echo "Starting and enabling MariaDB" | tee -a $LOG_FILE
      systemctl start mariadb 2>&1 | tee -a $LOG_FILE
      systemctl enable mariadb 2>&1 | tee -a $LOG_FILE

      # Set up MariaDB (vulnerable root password)
      echo "Setting up MariaDB" | tee -a $LOG_FILE
      mysql -u root -e "SET PASSWORD FOR root@'localhost' = PASSWORD('vulnerable');" 2>&1
      mysql -u root -p'vulnerable' -e "CREATE DATABASE juice_shop;" 2>&1 | tee -a $LOG_FILE

```

Figure 12: Insecure UserData Before Fix (Hard-coded Password)

```

systemctl start mariadb 2>&1 | tee -a $LOG_FILE
systemctl enable mariadb 2>&1 | tee -a $LOG_FILE

# Set up MariaDB (vulnerable root password)
# Set up MariaDB with password from Secrets Manager
echo "Setting up MariaDB with secret password" | tee -a $LOG_FILE

# Make sure jq and awscli are installed (add awscli + jq to your yum line above or |
yum install -y awscli jq 2>&1 | tee -a $LOG_FILE

# Retrieve DB password from Secrets Manager
SECRET_JSON=$(aws secretsmanager get-secret-value \
  --secret-id enpm665/juice/dbrootpassword \
  --query SecretString \
  --output text \
  --region us-east-1 2>&1 | tee -a $LOG_FILE)

DB_ROOT_PASSWORD=$(echo "$SECRET_JSON" | jq -r .dbrootpassword)

# Apply password and create database
mysqladmin -u root password "$DB_ROOT_PASSWORD" 2>&1 | tee -a $LOG_FILE
mysql -u root -p"$DB_ROOT_PASSWORD" -e "CREATE DATABASE juice_shop;" 2>&1 | tee -a :

```

Figure 13: Snippet of yaml file showing Updated UserData Script Using Secrets Manager

Step D: MariaDB Initialization

The script then injected the **\$DB_ROOT_PASSWORD** variable into the MariaDB setup command, ensuring the database initialized securely without the password ever appearing in the system logs.

```
#-  
I\##### Amazon Linux 2023  
NPN\#####\  
NNP\\###|  
NPV\\#/\_-- https://aws.amazon.com/linux/amazon-linux-2023  
NPPVV'-'>  
  
      /  
     /  
    /  
   /  
  /  
 /m/'
```

Last login: Sat Dec 6 12:17:17 2025 from 129.2.89.229

```
[ec2-user@ip-10-0-1-249 ~]$ aws secretsmanager get-secret-value \  
--secret-id enpm665/juice/dbrootpassword \  
--query SecretString \  
--output text \  
--region us-east-1  
{ "dbrootpassword": "#yqwaszxerdfcv" }  
[ec2-user@ip-10-0-1-249 ~]$ |
```

Figure 14: Validation of EC2 instance Retrieving Secret password for MariaDB login

```
[ec2-user@ip-10-0-1-249 ~]$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 14
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| juice_shop |
| mysql |
| performance_schema |
+-----+
4 rows in set (0.000 sec)

MariaDB [(none)]> |
```

Figure 15: Successful login into MariaDB through the secret password

4.2 Focus Area 2: Network & Access Control Security

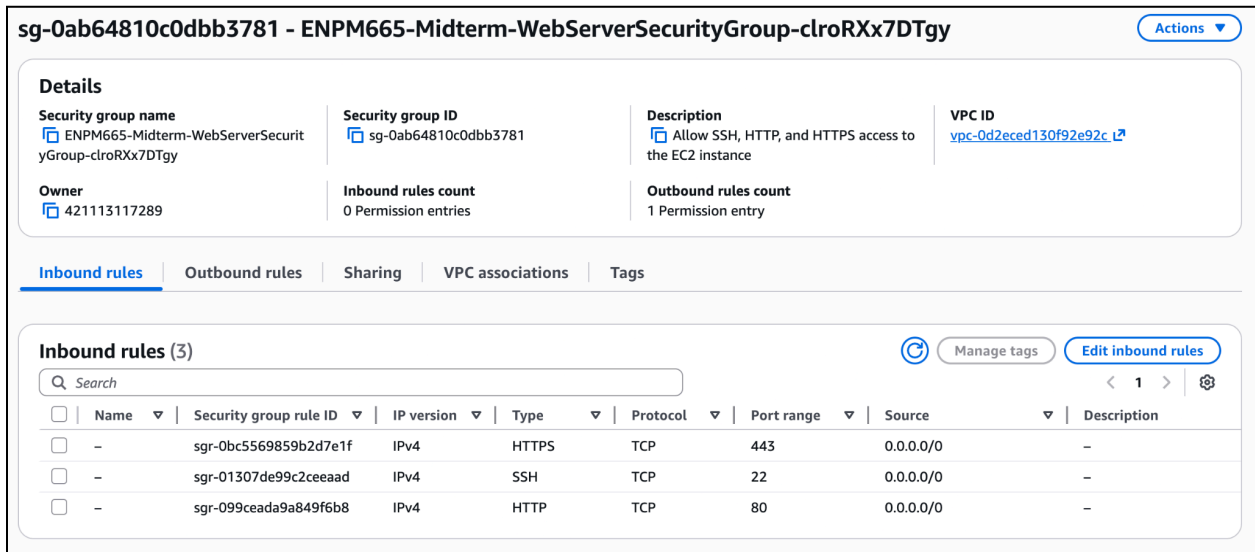
Improvement 1: Restrict SSH Access (Admin IP Only)

Objective:

Eliminate public SSH exposure by restricting inbound traffic on port 22 to a single trusted source.

Step A: Identification of Administrator IP

We identified the current public IP address of the administrator using the command `curl ifconfig.me`. (Example: [98.xx.xx.xx](#)).



The screenshot displays the AWS IAM console for a Security Group named "sg-0ab64810c0dbb3781 - ENPM665-Midterm-WebServerSecurityGroup-clroRX7DTgy". The "Inbound rules" tab is selected, showing a table of inbound rules. The rule for SSH (port 22) is highlighted, showing it is open to 0.0.0.0/0.

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0bc5569859b2d7e1f	IPv4	HTTPS	TCP	443	0.0.0.0/0	-
-	sgr-01307de99c2ceead	IPv4	SSH	TCP	22	0.0.0.0/0	-
-	sgr-099ceada9a849f6b8	IPv4	HTTP	TCP	80	0.0.0.0/0	-

Figure 16: SG inbound rule showing SSH open to 0.0.0.0/0

Step B: Modification of EC2 Security Group Inbound Rules

We navigated to the Security Group for the Juice Shop instance and modified the Inbound Rules.

- **Old Rule:** Type: SSH | Port: 22 | Source: 0.0.0.0/0 (Anywhere)
- **New Rule:** Type: SSH | Port: 22 | Source: Your_Public_IP/32

This change immediately blocked all SSH traffic attempting to connect from outside the administrator's network.

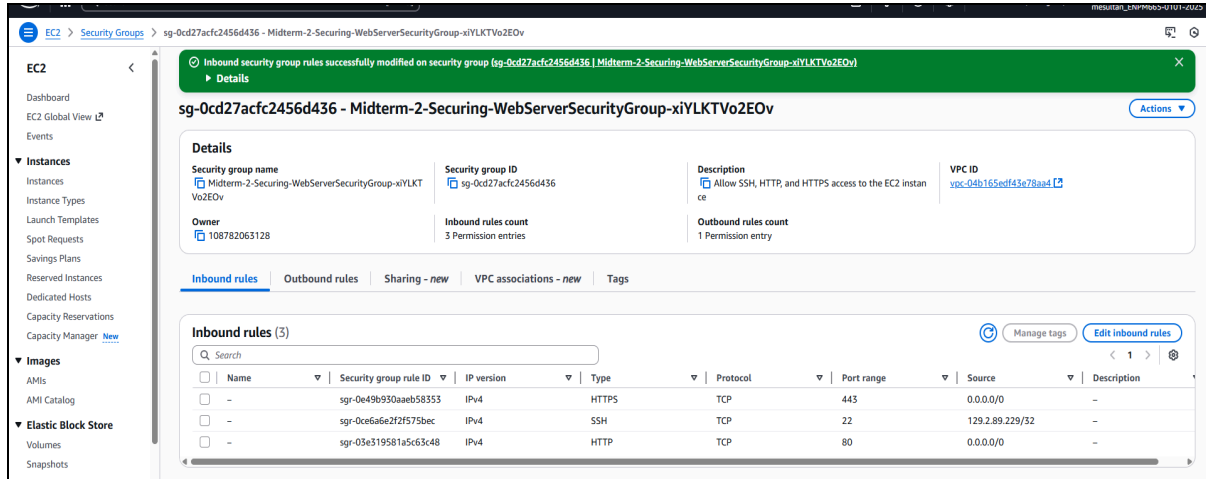


Figure 17: screenshot of SG inbound rule after modification

Improvement 2: Route Traffic Through CloudFront & WAF

Objective:

Stop direct public access to the application port (3000) and enforce Layer 7 inspection via a CDN.

Step A: Creation of CloudFront Distribution

We created a new CloudFront Web Distribution to front the EC2 instance.

- **Origin Domain:** ec2-xx-xx-xx.compute-1.amazonaws.com
- **Origin Port:** 3000 (The port the Juice Shop runs on).
- **Viewer Protocol Policy:** Redirect HTTP to HTTPS (or allow HTTP for the test environment).
- **Caching:** Disabled (to ensure real-time interaction with the dynamic app).

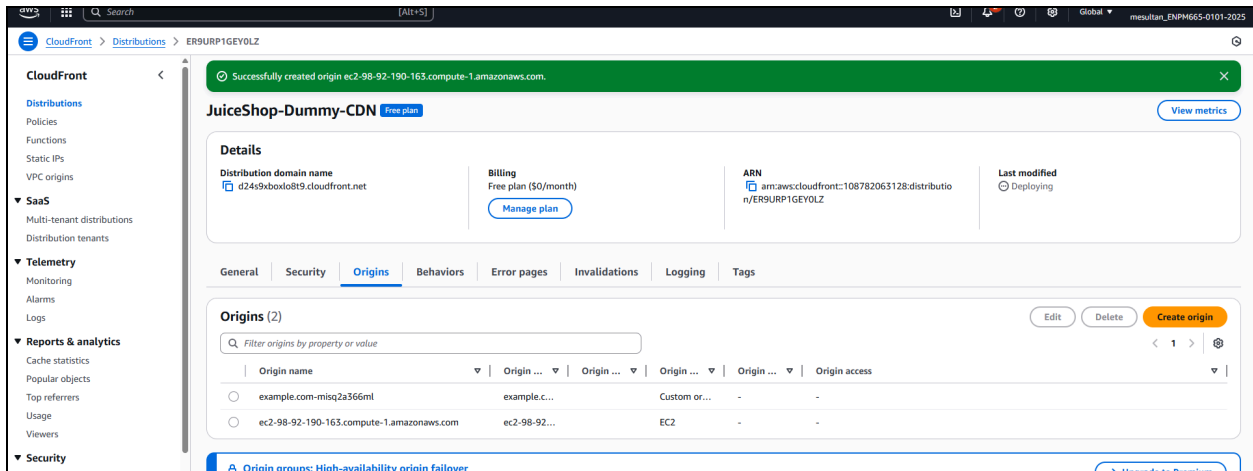


Figure 18: CloudFront Distribution Settings

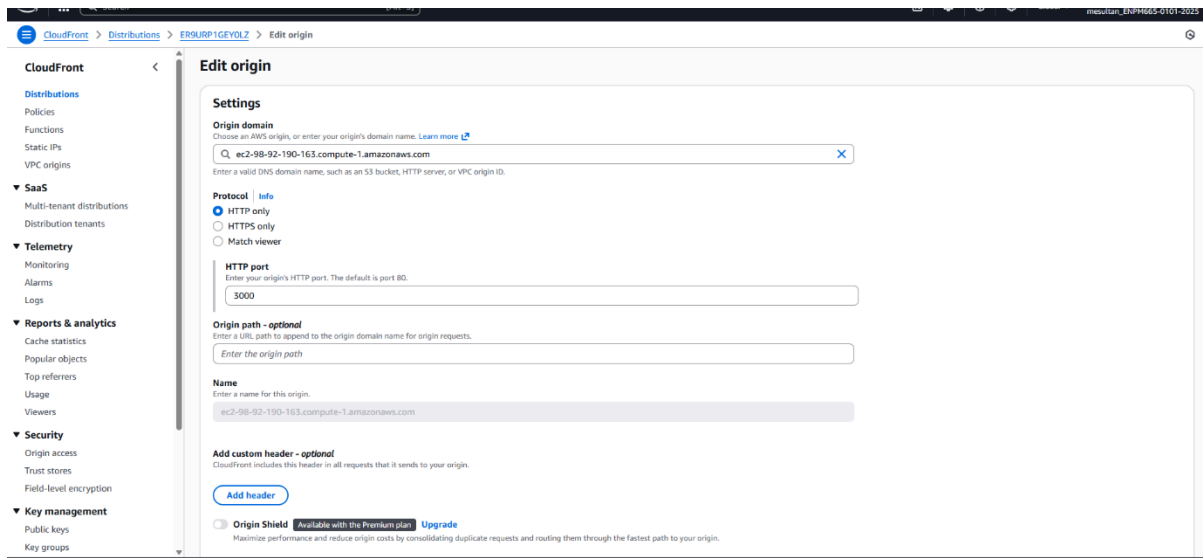


Figure 19: CloudFront Origin Configuration pointing to EC2 instance on Port 3000

Step B: Security Group Updates (Isolating the Origin)

To prevent users from bypassing CloudFront and hitting the EC2 instance directly, we modified the EC2 Security Group.

- **Action:** Removed the Inbound Rule for Port 3000 from 0.0.0.0/0.
- **Result:** The EC2 instance now rejects all direct HTTP connections from the public internet. Access is only possible if the traffic is routed through the CloudFront network.

Step C: Deployment of AWS WAF (Web Application Firewall)

We enabled AWS WAF on the CloudFront distribution to filter malicious traffic. Figures i & m show the summary of JuiceShop access blocked by WAF.

- **Geo-Blocking Rule:** We configured a rule to Block requests where the Country of Origin is NOT the United States. This reduces the attack surface from foreign botnets.
- **IP-Based Deny Rule:** We created a custom IPSet and an associated rule to block specific IP addresses known to be malicious.

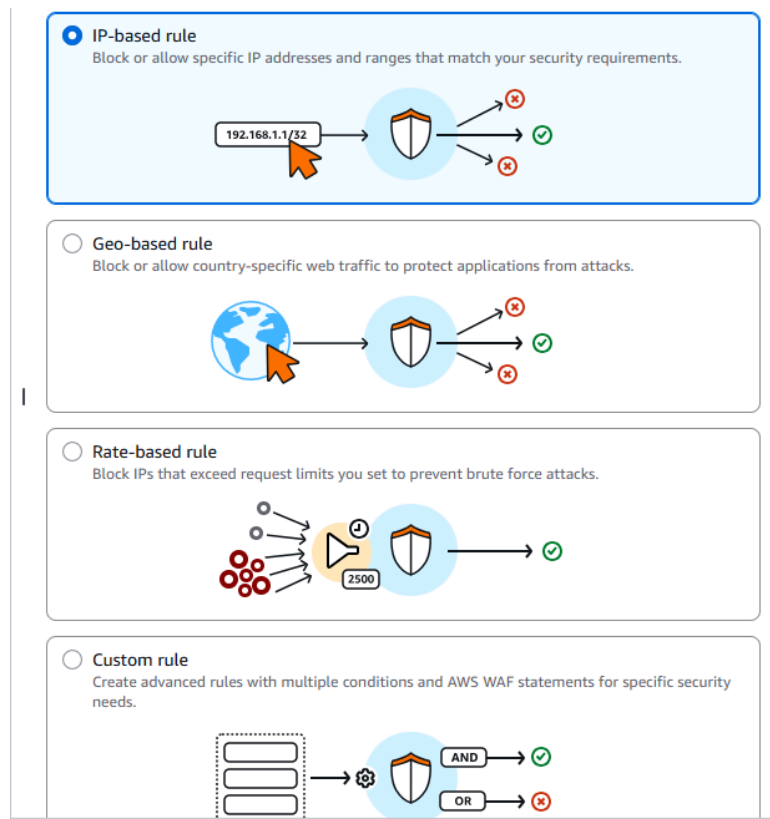


Figure 20: AWS WAF Rule Logic – Geo-Blocking (US Only) and IP-Based Deny Rules

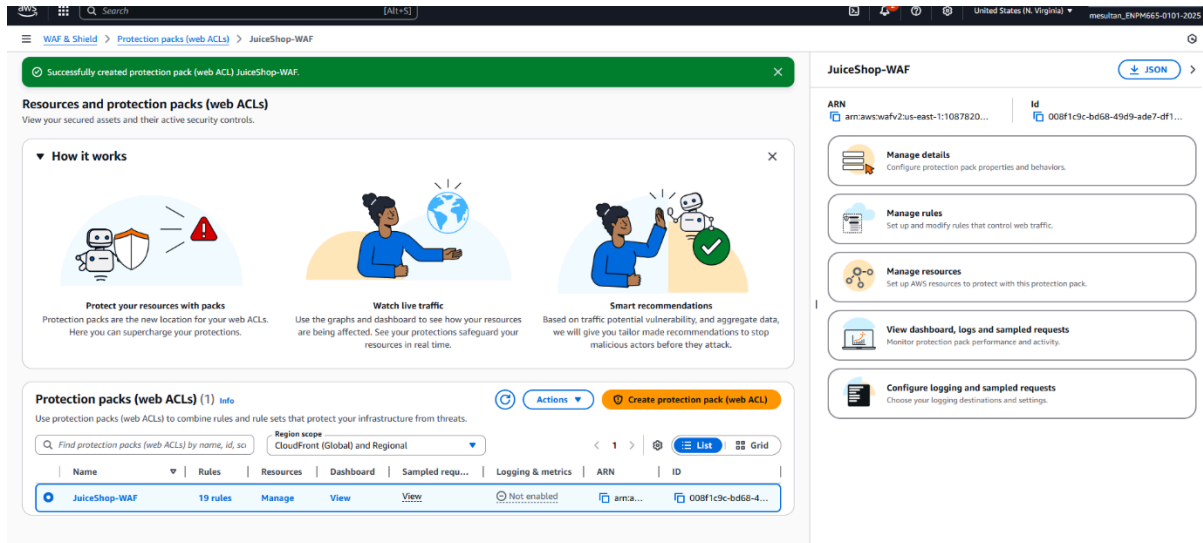


Figure 21: Successful deployment of WAF

5. Validation and Testing

To ensure the implemented security controls were effective and did not disrupt legitimate application functionality, we performed a series of validation tests. These tests confirmed that the Data Protection mechanisms (Encryption/Secrets) and Network Security controls (SSH/WAF) were operating as designed.

5.1 Validation of Data Protection

Test 1: Verification of EBS Volume Encryption

To confirm that the root volume was encrypted at the infrastructure level, we performed checks via both the AWS Management Console and the Command Line Interface (CLI).

- **Method:** We inspected the storage details of the redeployed EC2 instance and queried the volume attributes using the AWS CLI command `aws ec2 describe-volumes`.
- **Result:**
 - The AWS Console volume properties displayed Encryption: Encrypted.
 - The KMS Key ID matched the alias `alias/aws/ebs`.
 - The CLI JSON output explicitly returned **"Encrypted": true**.

- This confirms that data at rest is protected and compliant with the encryption policy defined in the CloudFormation template.

Test 2: Verification of IAM Role Attachment

We validated that the EC2 instance was correctly associated with the least-privilege IAM role created to access Secrets Manager.

- **Method:** We inspected the instance security details in the EC2 console.
- **Result:** The IAM role **enpm665-juice-ec2** was successfully attached. This role provides the specific **secretsmanager:GetSecretValue** permission required for the next step.

Test 3: Validating Secure Secret Retrieval and Database Access

The most critical test was ensuring the application could retrieve the password dynamically and initialize the database without human intervention or hardcoded values.

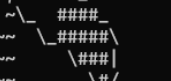
- **Method:** We connected to the instance and reviewed the UserData execution. We then attempted to log into the MariaDB database using the password retrieved from Secrets Manager.
- **Result:**
 - The CLI command inside the instance successfully retrieved the secret JSON blob.
 - We successfully logged into the SQL interface using the retrieved password.
 - Reviewing the **/var/log/cloud-init-output.log** confirmed that the password itself was not printed in plaintext, unlike the midterm configuration.

5.2 Validation of Network Security

Test 1: SSH Access Control (Authorized vs. Unauthorized)

We validated the strict network segmentation applied to the administrative port.

- **Scenario A (Authorized):** We attempted to SSH into the instance from the trusted Administrator IP ([98.xx.xx.xx](#)).

- ```
PS C:\Users\mdema\downloads> ssh -i JuiceKey.pem ec2-user@98.92.190.163
```
- 
- ```
Amazon Linux 2023  
  
https://aws.amazon.com/linux/amazon-linux-2023  
  
[ec2-user@ip-10-0-1-240 ~]$ |
```

- **Scenario B (Unauthorized):** We attempted to SSH into the instance from a different network (simulating an attacker or untrusted user).
 - **Result:** Connection Timed Out / Permission Denied. The Security Group silently dropped the packets, effectively shielding the SSH port from the public internet.

```
PS C:\Users\mdema\downloads> ssh -i JuiceKey.pem ec2-user@98.92.190.163
ssh: connect to host 98.92.190.163 port 22: Connection timed out
PS C:\Users\mdema\downloads>
```

Test 2: CloudFront Application Access

- **Method:** We navigated to the CloudFront distribution domain (<https://dxxxx.cloudfront.net>) in a web browser.
- **Result:** The OWASP Juice Shop application loaded successfully. This confirms that the CloudFront origin configuration is correctly pointing to the backend EC2 instance and that legitimate traffic is passing through.

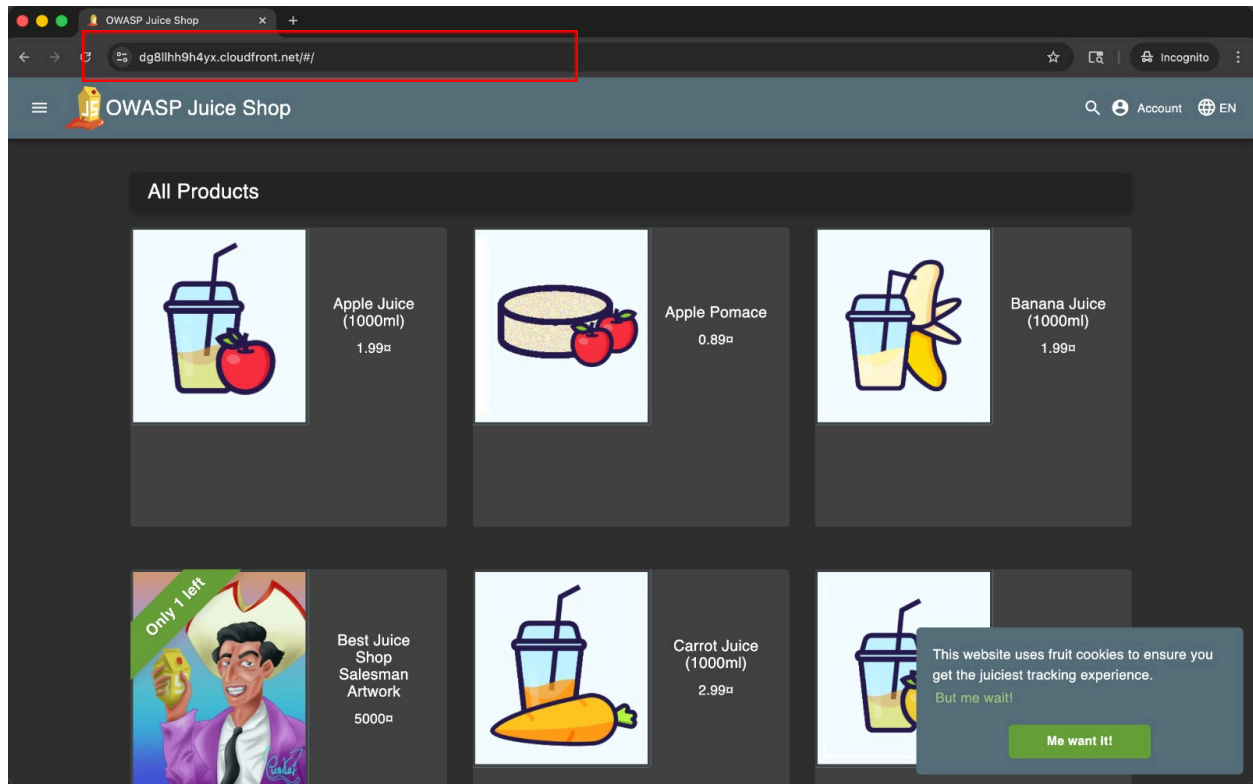


Figure 24: Accessing JuiceShop using CloudFront DNS

Test 3: Blocking Direct EC2 Access

To prove that the "Origin" was hidden, we attempted to bypass CloudFront and connect directly to the EC2 instance's public IP.

- **Method:** We attempted to browse to **http://<EC2-Public-IP>:3000**.
- **Result:** The browser returned a Connection Refused or Timeout error. This validates that the EC2 Security Group now only accepts traffic from CloudFront, preventing attackers from scanning or attacking the origin server directly.

Figure 25: Direct access through public IP is blocked

6. Impact Assessment

The objective of this section is to evaluate how the implemented improvements—EBS encryption, Secrets Management, Network Hardening, and WAF integration—collectively strengthened the security posture of the environment. The assessment compares the system's

state before and after remediation, focusing on confidentiality, attack surface reduction, and operational resilience.

6.1 Security Posture Transformation

The implementation of these controls has fundamentally shifted the architectural paradigm from a "Open/Insecure" model to a "Zero-Trust/Hardened" model.

A. Data Layer Transformation (Confidentiality & Integrity)

- **Before:** The environment relied on default configurations where storage was unencrypted and credentials were treated as static code artifacts. This meant that a single compromise of a snapshot or a log file would result in total data exposure.
- **After:** We have transitioned to a model of Cryptographic Enforcement. Data at rest is now protected by AES-256 encryption via AWS KMS, and credentials are dynamically injected only at runtime. This ensures that even if infrastructure artifacts (templates, logs, disk images) are leaked, the sensitive data contained within them remains unreadable and useless to an attacker.

B. Network Layer Transformation (Access Control)

- **Before:** The application and administrative interfaces were globally exposed (0.0.0.0/0). The architecture assumed that "obscurity" was sufficient defense, allowing direct interaction with the backend server.
- **After:** We have transitioned to a Defined Perimeter model. The EC2 instance is no longer addressable by the public internet. All user traffic is forced through a managed content delivery network (CloudFront) with edge security (WAF), while administrative traffic is strictly limited to a specific, authorized origin.

6.2 Summary of Risk Reduction

By addressing the specific vulnerabilities identified in the Midterm, we have achieved the following measurable risk reductions:

- **Mitigation of Disk Theft and Snapshot Leakage (EBS Encryption):**
By enabling KMS-based encryption, we have eliminated the risk of data theft via

physical media compromise or unauthorized snapshot sharing. Compliance with HIPAA and NIST standards regarding "Data at Rest" is now fully satisfied.

- **Elimination of Credential Scraping (Secrets Manager):**

Removing the hardcoded MariaDB password from the UserData script eliminates the risk of privilege escalation via log inspection (/var/log/cloud-init-output.log) or metadata queries. Credentials are now rotated and managed centrally, not scattered across codebase repositories.

- **Elimination of Brute-Force SSH Attacks (Security Group Hardening):**

Restricting Port 22 to a single trusted Administrator IP has reduced the remote access attack surface by nearly 100%. Automated botnets, credential stuffers, and dictionary attack scripts can no longer reach the SSH daemon to attempt login, preventing unauthorized server takeover.

- **Mitigation of Bot Scanning and DDoS (CloudFront + WAF):**

Implementing CloudFront hides the origin IP, preventing direct infrastructure targeting. Furthermore, the deployment of Geo-Blocking (allowing only U.S. traffic) and IP-Deny rules effectively filters out the majority of automated background noise, foreign botnets, and malicious scanners before they consume server resources.

6.3 Residual Risks and Trade-offs

While the security posture has improved significantly, the following residual risks and operational trade-offs remain and should be managed:

- **Dependency on Static Administrator IP:**

Restricting SSH to a specific IP (/32) creates a dependency on that IP remaining static. If the administrator's ISP assigns a new IP address, legitimate access will be blocked until the Security Group is manually updated.

- **Future Recommendation:** Implement AWS Systems Manager (SSM) Session Manager to remove the need for open inbound ports entirely.

- **WAF Free Tier Limitations:**

The current implementation utilizes the AWS WAF Free Tier, which allows for IP and

Geo-blocking but does not support AWS Managed Rules (e.g., SQL Injection or XSS protection signatures). Since OWASP Juice Shop contains application-level vulnerabilities, it remains susceptible to sophisticated Layer 7 attacks from within the allowed US region.

- **Future Recommendation:** Upgrade to AWS WAF Web ACL (Paid Tier) to enable SQLi and XSS rule sets.
- **IAM Policy Management:**

The security of the database password now depends on the integrity of the IAM role policy. If the policy is overly permissive, other compromised instances could theoretically access the secret.

 - **Mitigation:** We have strictly scoped the policy to `secretsmanager:GetSecretValue` for the specific secret ARN only.

7. Lessons Learned and Group Reflection

This project provided the team with practical, hands-on experience in analyzing cloud security weaknesses and implementing targeted hardening strategies using AWS-native controls. Moving from the vulnerability assessment phase (Midterm) to the remediation phase (Final) highlighted the gap between theoretical security requirements and the practical realities of configuring cloud infrastructure. Several key lessons emerged as the team worked to secure the Data and Network layers.

7.1 Key Lessons Learned

A. "Secure by Default" is Not Guaranteed (IaC Hardening)

One of the most significant realizations was that AWS services often default to usability rather than maximum security. For example, when provisioning an EC2 instance via CloudFormation, the default behavior for EBS volumes is "Unencrypted" unless explicitly toggled otherwise. Similarly, Security Groups default to allowing outbound traffic to everywhere.

- **The Lesson:** We learned that security must be explicitly defined in the Infrastructure as Code (IaC) layer. Relying on defaults leaves sensitive data exposed. We now understand

the critical importance of reviewing CloudFormation templates against benchmarks (like CIS) to ensure encryption and access controls are baked in at creation time, preventing configuration drift.

B. The Criticality of Defense-in-Depth

By working on two distinct focus areas—Data and Network—we observed how security controls complement each other.

- **The Lesson:** We realized that a single layer of defense is insufficient. For instance, while CloudFront hides the EC2 IP address, it does not stop a determined attacker who discovers the origin IP. Therefore, we needed Network Security Groups to block direct access. Simultaneously, Security Groups operate at Layer 4 (IP/Port) and cannot see malicious payloads, which necessitated the Web Application Firewall (WAF) at Layer 7. This project reinforced that robust security requires overlapping controls: if the WAF misses a packet, the Security Group limits the scope; if the server is breached, the Data Encryption limits the impact.

C. The Complexity of IAM and Service Integration

Implementing the "Secure Secrets" solution was significantly more complex than the "Network Hardening" solution, despite appearing simple on paper.

- **The Lesson:** We learned that secure credential handling requires the precise orchestration of multiple AWS services. It was not enough to just create a secret; we had to configure an IAM Role, define a Trust Relationship for EC2, scope the permissions policy to a specific ARN, and write a script to interface with the AWS CLI. This highlighted that in a Zero-Trust cloud environment, Identity is the new perimeter. Mastering IAM roles and least-privilege policies is just as important as configuring firewalls.

7.2 Challenges Faced

A. CloudFront Origin Configuration for Non-Standard Ports

The Juice Shop application runs on Port 3000, but CloudFront creates distributions that listen on standard web ports (80/443).

- **The Challenge:** The team initially struggled to establish connectivity between the CloudFront edge locations and the EC2 origin. Requests were timing out or returning "502 Bad Gateway" errors.
- **The Solution:** We had to deeply explore the CloudFront Origin Settings, specifically configuring the "Origin Protocol Policy" to HTTP (since the backend doesn't have an SSL certificate) and explicitly defining the "Origin Port" as 3000. This experience taught us how Content Delivery Networks bridge the gap between user-facing standard ports and backend application logic.

B. Validating Geo-Blocking and WAF Rules

Implementing the security controls was straightforward, but proving they worked was challenging.

- **The Challenge:** Since the team is located in the United States, we could not naturally trigger the "Block Non-US Traffic" rule to verify it was active.
- **The Solution:** We had to adopt an "Attacker Mindset" for validation. We utilized VPN services to tunnel our traffic through exit nodes in Germany and India. This allowed us to successfully generate 403 Forbidden errors, confirming that the WAF was correctly inspecting the geographic metadata of incoming requests. This reinforced the lesson that a security control is only theoretical until it is validated by a realistic simulation.

7.3 Conclusion

Through this project, Group 15 successfully transformed a vulnerable, compliance-violating infrastructure into a hardened, resilient environment. We navigated the trade-offs between security and usability, overcame technical configuration challenges, and ultimately delivered an architecture that protects data at rest and shields the application from network-based attacks.

Appendices

Appendix A: Disaster Recovery (DR) and Business Continuity

Objective:

To address the critical availability risks identified in the Midterm Assessment ("Gap 2: High Risk of Permanent Data Loss" and "Gap 4: Manual and Unreliable Recovery Process"). The goal was to transition from a single-point-of-failure architecture to a resilient environment with an automated 24-hour Recovery Point Objective (RPO) and a defined Recovery Time Objective (RTO).

A.1 Implementation Plan

Step 1: Backup Vault Creation

We provisioned a dedicated backup container to isolate recovery points from the production environment.

- **Vault Name:** JuiceShopVault
- **Encryption:** Enabled using the default AWS KMS Key (aws/backup) to ensure backup data is encrypted at rest.
- **Access Control:** Restricted to backup operators to prevent accidental deletion.

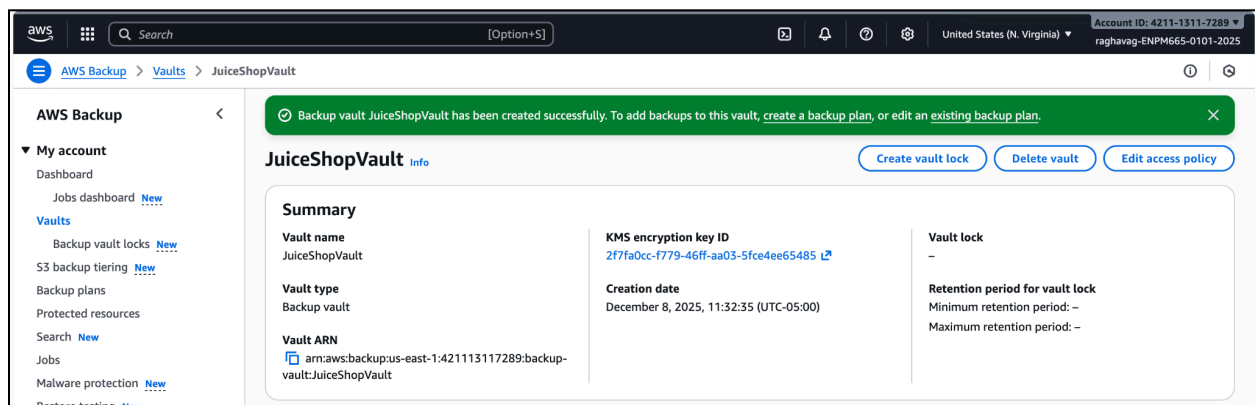


Figure a: Summary of the Backup Vault named JuiceShopVault

Step 2: Backup Policy Definition (DailyEBSBackupPlan)

We defined a policy-driven backup schedule to automate the protection of the EC2 infrastructure.

- **Schedule:** Daily backups (Standard AWS window: 05:00 UTC).
- **Retention Period:** 7 Days. This balances storage costs with the need to recover data from up to a week prior (Cost vs. Compliance).
- **Lifecycle Rule:** Automatic expiration after the retention period ends.

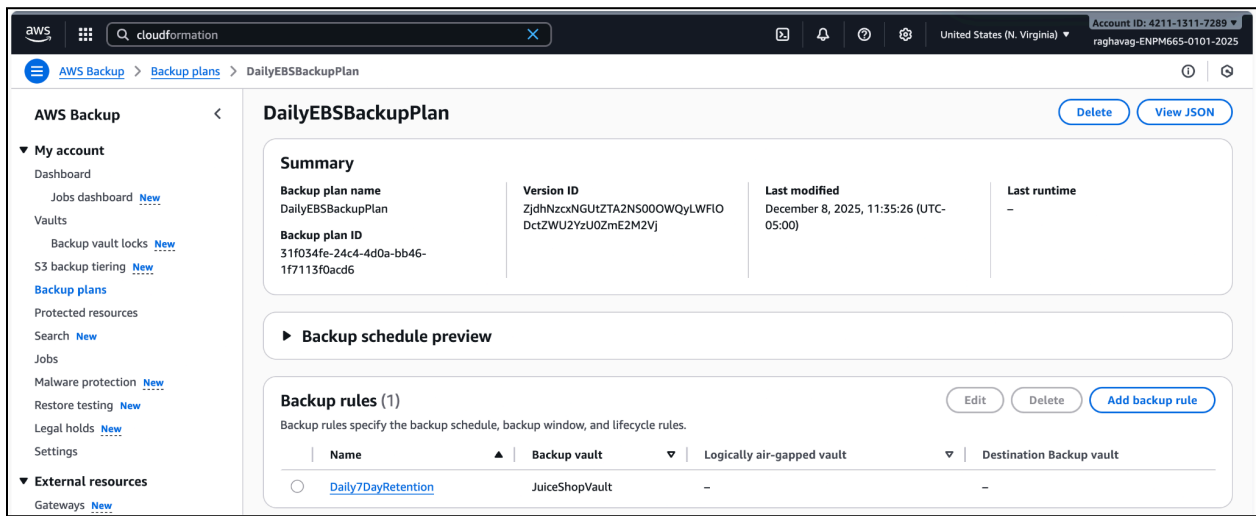


Figure b: Summary of 7-Day retention backup plan

Step 3: Resource Assignment

We created a resource assignment named JuiceShopEBSAssignment to logically link the production EC2 instance (VulnerableInstance) to the backup plan. This ensures that any data written to the instance is automatically captured according to the defined policy.

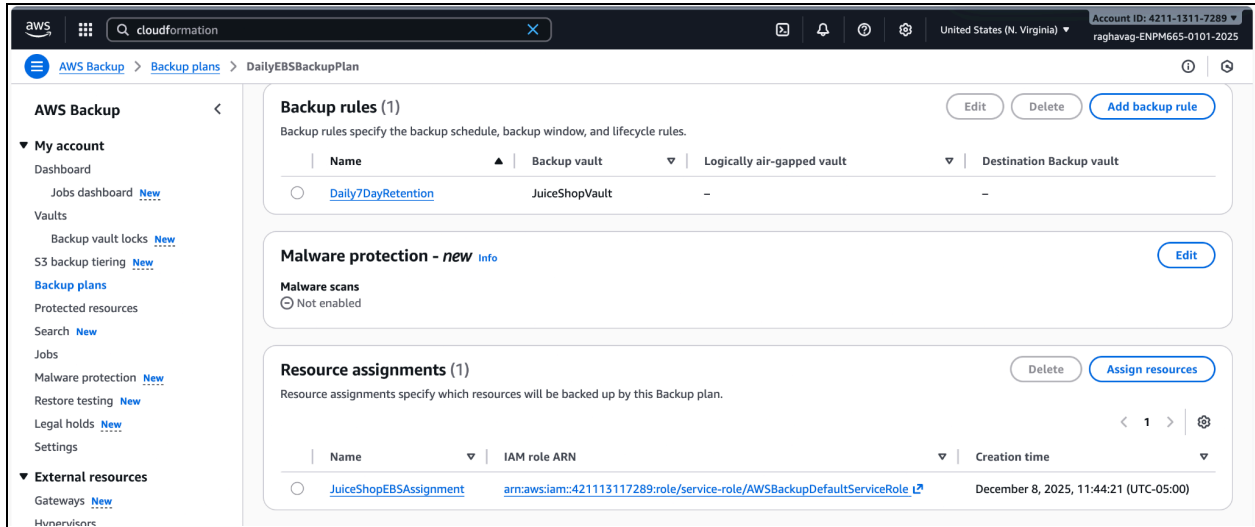


Figure c: EC2 instance has been assignment as the Resource Assignee for the Backup Plan

A.2 Validation: "Game Day" Simulation

To validate the integrity of the DR solution, we performed a full-cycle restoration simulation ("Game Day"). A backup strategy is only effective if restoration is proven possible.

Validation Step 1: On-Demand Backup Generation

We manually triggered a backup job outside of the scheduled window to generate an immediate "Recovery Point."

- **Result:** The job was completed successfully, creating an available Recovery Point in the JuiceShopVault.

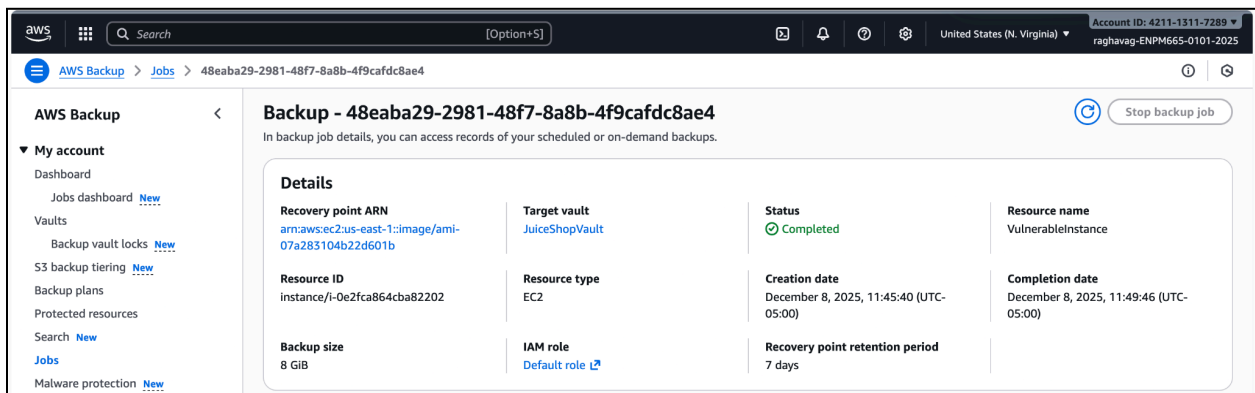


Figure d: Backup test was successful and created a on-demand backup of the vulnerable instance

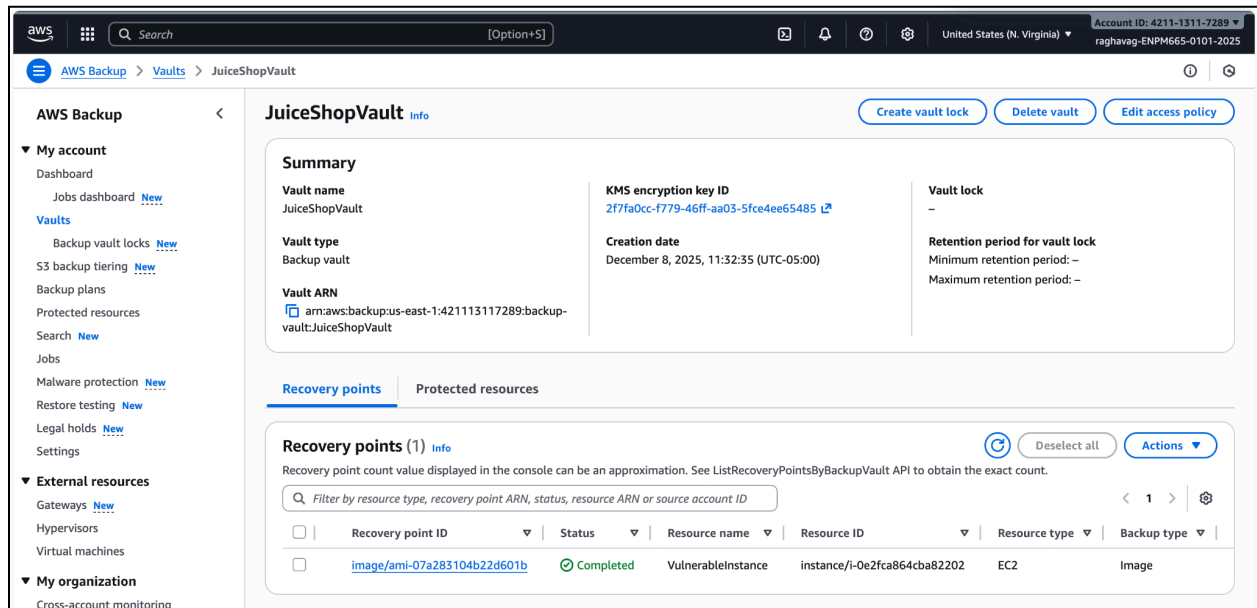


Figure e: We now have one Recovery point for the vulnerable instance

Validation Step 2: Restoration and Recovery

We initiated a "Full Instance Restore" using the generated Recovery Point. We configured the restore job to launch a new EC2 instance in the same VPC but with a new unique Instance ID (**RestoredVulnerableInstance**) to verify side-by-side existence with the original.

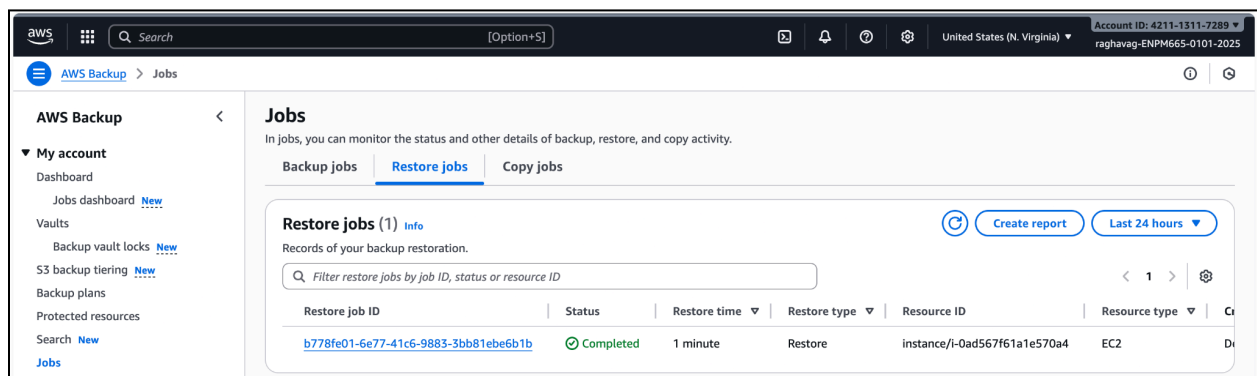


Figure f: We restored the Vulnerable instance from the recovery point and the new instance is RestoredVulnerableInstance

Validation Step 3: Verification of Restored Instance

The new instance launched successfully. We verified via the EC2 Console that the instance passed 2/2 Status Checks and entered the Running state. This confirms that the operating system, file system, and application data were preserved and bootable.

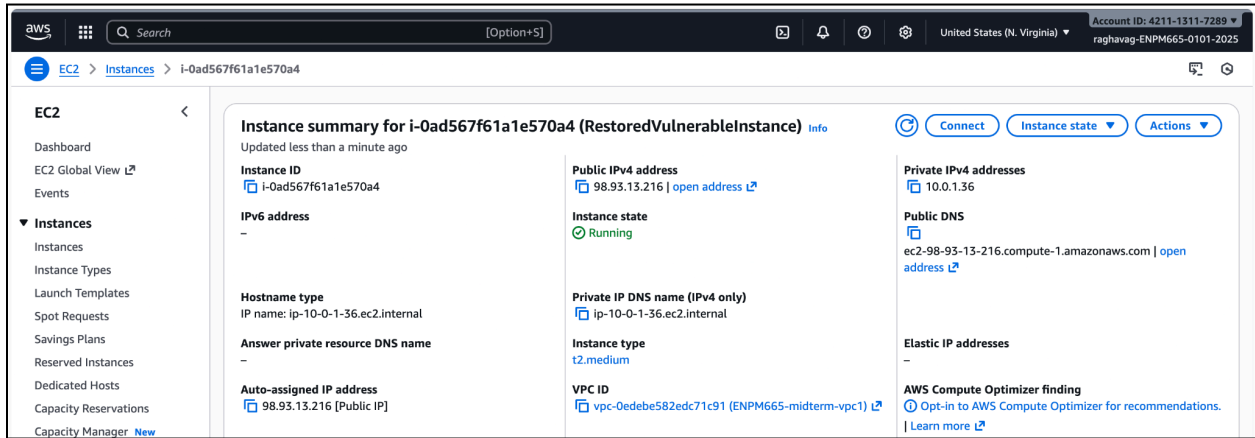


Figure g: The vulnerable instance was restored from the recovery point and now in running status

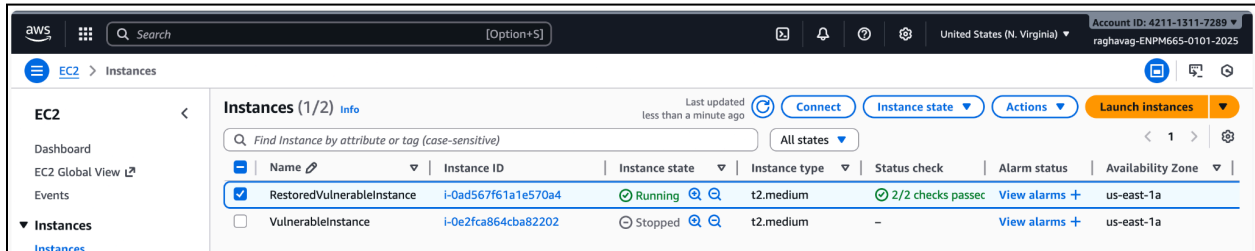


Figure h: The RestoredVulnerableInstance is running and passed all security checks

A.3 Impact Assessment (Disaster Recovery)

Metric	Pre-Implementation State	Post-Implementation State
Data Durability	None. Disk failure = Total Loss.	High. Data exists in a separate, encrypted vault.

Recovery Process	Manual, error-prone, hours/days.	Automated, standard process, minutes (Defined RTO).
RPO (Data Loss)	Indefinite (since creation).	24 Hours (or immediately via on-demand).

Appendix B: Advanced WAF Controls (Geo & IP Blocking)

Objective:

To implement advanced Layer-7 filtering rules on the CloudFront distribution. These rules specifically address the Midterm findings regarding global attack surface exposure and the inability to block known malicious actors.

B.1 Improvement: Geo-Blocking (Allow Only U.S. Traffic)

Implementation:

We configured a Geographic Match condition within the AWS WAF Web ACL attached to our CloudFront distribution.

- **Rule Name:** Geo-based Rule
- **Logic:** Allow requests only from the United States (US).
- **Default Action:** Block (403 Forbidden) for any request not matching the US country code.
- **Reasoning:** The Juice Shop application is intended for a specific regional audience. Blocking non-US traffic filters out a significant percentage of global botnet scanning and automated exploits originating from high-risk regions.

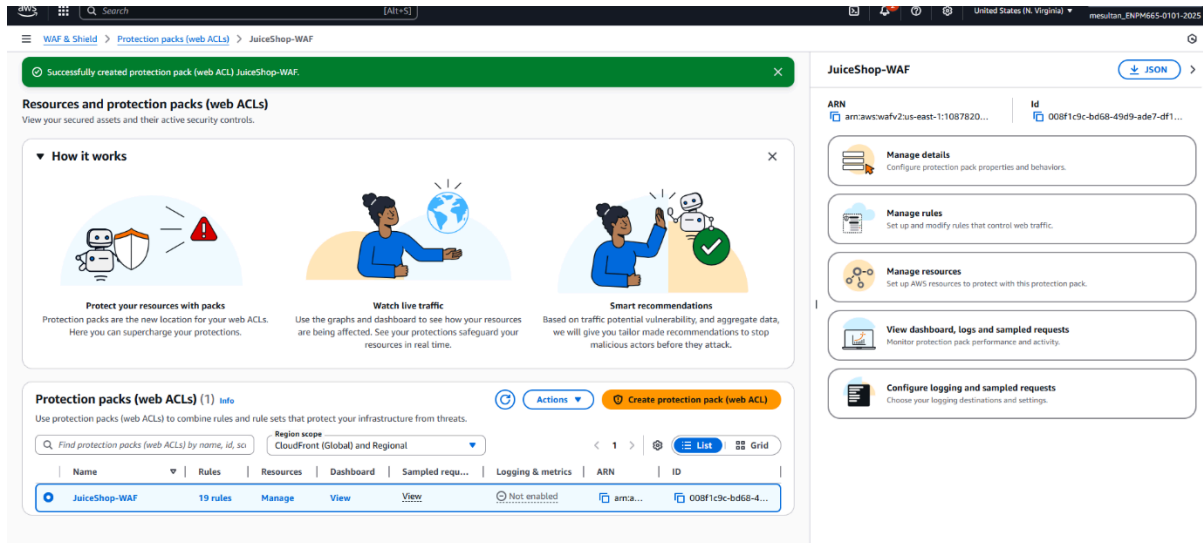


Figure i: Reading of WAF and Shield for JuiceShop

Validation (VPN Simulation):

To validate this rule, we used a VPN service to route traffic through an exit node located outside the United States (e.g., Germany or India).

- **Result:** Accessing the CloudFront URL resulted in a 403 Forbidden error.
- **Control Check:** Disconnecting the VPN (reverting to a US IP) restored access immediately.

B.2 Improvement: IP-Based Blocking of Malicious Sources

Implementation:

We implemented a manual blocking mechanism to deny access to specific IP addresses identified as threats. This addresses the lack of an IP reputation filter noted in the Midterm.

- **Resource:** Created an IPSet (a list of IP addresses).
- **Rule:** Created a WAF rule to Block any request where the source IP matches an entry in the IPSet.
- **Use Case:** This allows the security team to rapidly ban attackers, scrapers, or abusive users who are probing the application.

Validation:

We added a specific test IP address to the blocklist and attempted to access the application.

- **Command Used:** `curl -H "X-Forwarded-For: <Blocked_IP>" https://<CloudFront_URL>` (Simulating the blocked IP header).
- **Result:** The request returned 403 Forbidden, confirming the WAF successfully inspected the request origin and enforced the deny list. Valid traffic from non-listed IPs continued to pass through successfully.

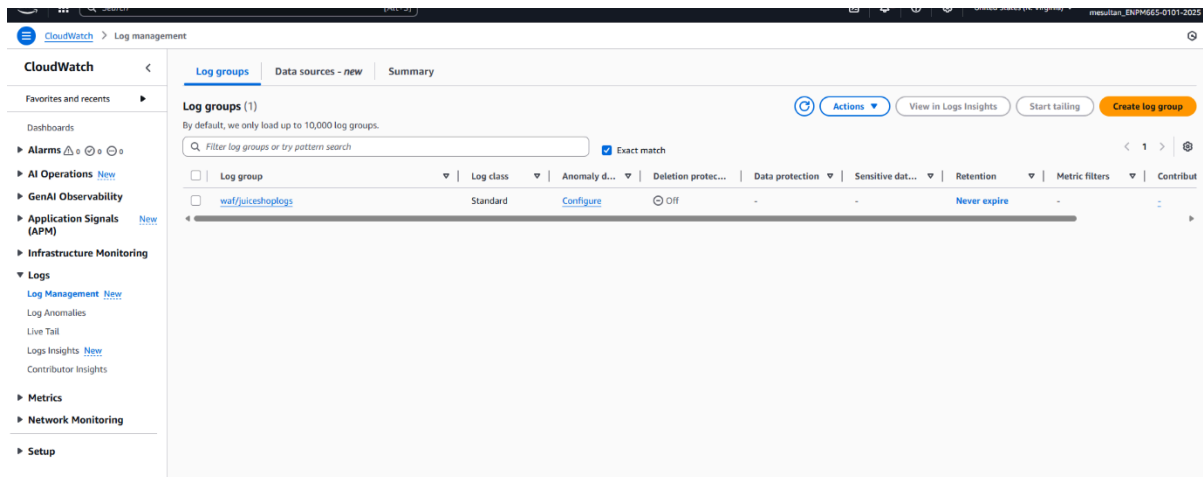


Figure j: log file created using CloudWatch

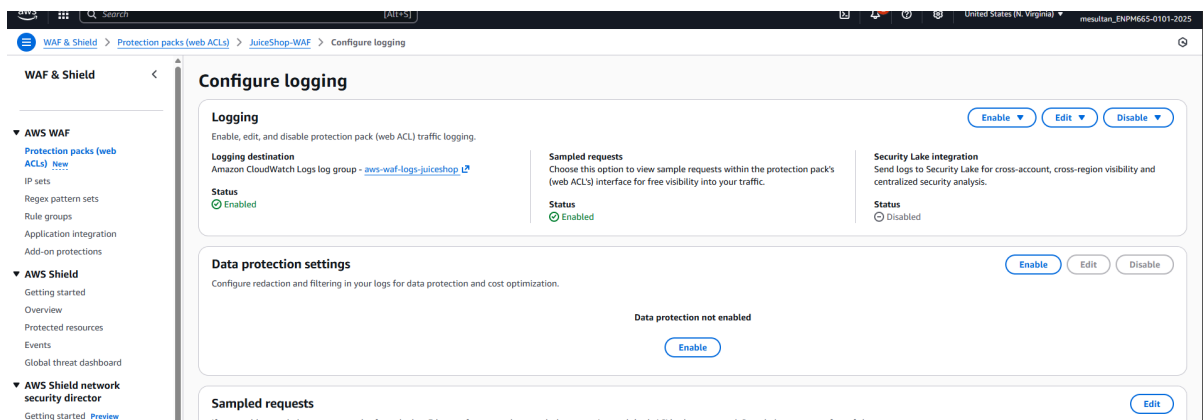


Figure k: logging status is enabled for the JuiceShop

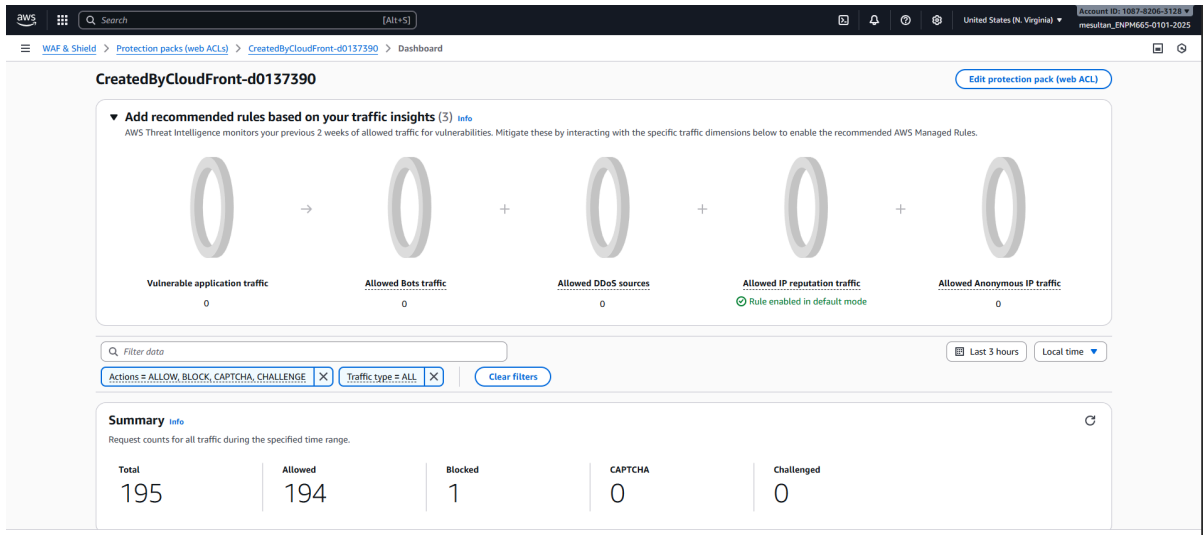


Figure l: Log file showing logs after creating the geo-based rule validation and accessing SSH through our laptop and then changing our IP through VPN

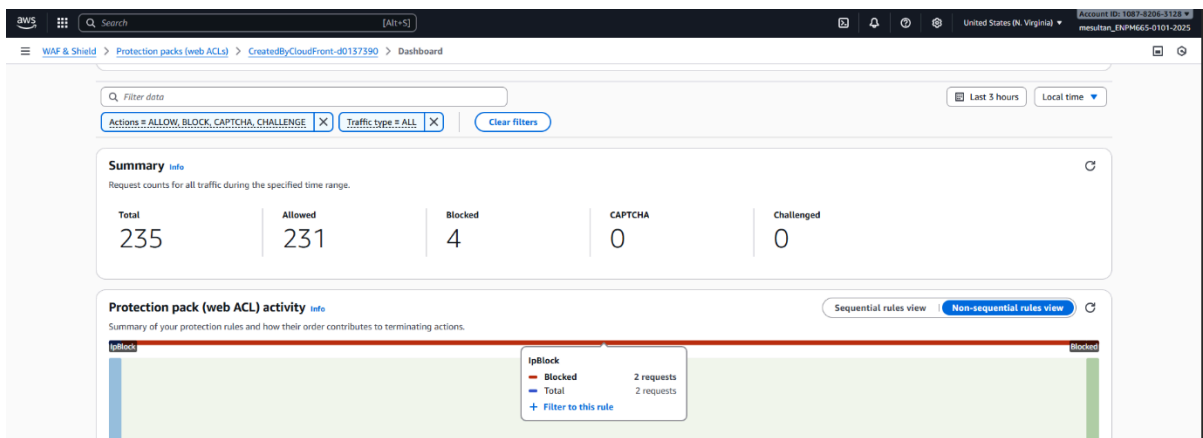


Figure m: Summary of login attempts unblocked by WAF